LPC'25 X86 Micro-conference

Leveraging New GPRs in the Kernel

Chang Bae, Intel



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Topics

- Advanced Performance Extensions (APX)
- In-kernel usage scenarios
- Enabling approaches

Topics

- Advanced Performance Extensions (APX)
  - Brief introduction
  - Toolchain status
- In-kernel usage scenarios
- Enabling approaches

APX: Extended GPRs

- APX introduces register extension
  - EGPRs: R16-R31
  - Volatile registers ⇦ ELF x86-64 ABI [repo]
  - XSTATE #19 ⇨ userspace support (v6.16 or later)
- In-Kernel: unprecedented expansion
  - First GPR architecture change since x86-64 introduction
  - Other architectures have established that amount of GPRs once (so far)

APX: EGPR Encoding

- REX2 prefix extends register identifiers
  - Add a 5th bit of REX **`R/X/B`** fields

REX2	Payload byte							
`0xD5`	`M0`	`R4`	`X4`	`B4`	`W`	`R3`	`X3`	`B3`

    - **`M0`** - legacy opcode map 0 or 1
- EVEX prefix also extended
  - Similar register bit extension semantics

APX: Additional Features

- Three-operand format with explicit destination register
  - New Data Destination (**NDD**)

Existing form	NDD extension
`INC r/m`	`INC ndd, r/m`
`SUB r/m, imm`	`SUB ndd, r/m, imm`
`SUB r/m, reg`	`SUB ndd, r/m, reg`
`SUB reg, r/m`	`SUB ndd, reg, r/m`

  - Encoded via an EVEX prefix bit

APX: Additional Features* (cont.)

- Register save/restore optimization
  - **`PUSH2/POP2`**: handle two registers per instruction
  - **`POP`** hint bit:
    - Mark **`POP`** paired with a preceding **`PUSH`**
    - Reduces memory traffic
- Extension to conditional instructions
  - **`CCMP`** / **`CTEST`** for broader if-conversion

* APX Architecture Specification has more details.

Compilers for APX

- GCC
  - V14 (2024): **`-mapxf`**, EGPR, and **`PUSH/POP`** hint support
  - V15 (2025): **`CCMP/CTEST`** and additional APX features



- LLVM
  - V18 (2024): **`-mapxf`** support



Topics

- Advanced Performance Extensions (APX)
- In-kernel usage scenarios
- Enabling approaches

In-Kernel Usage Scenarios

- Library code may benefit from extra scratch registers
  - Handwritten assembly example [Link]
- EGPRs may reduce kernel stack usage
  - GCC *static* stack analysis estimates ~3% reduction tree-wide
- Compilers may pick up APX features at scale
  - E.g. **`POP`** hint, **`CCMP/CTEST`**, or **`NDD`**

Topics

- Advanced Performance Extensions (APX)
- In-kernel usage scenarios
- Enabling approaches
  - Context switching: `XSAVE` vs. entry point
  - Application method: assembly vs. compilation
  - Application scope: limited vs. wide-range

Context Switching

- **`XSAVE`**
  - Current, established mechanism ⇨ simple
  - ABI defined by **`XSAVE`** format
- Entry-point switching
  - Follow legacy GPR handling
  - ABI considerations ⇨ complex
    - Extending to **`pt_regs`**? (Likely, not.)
    - Userspace exposure via **`XSAVE`**

Application Method

- **`XSAVE`** ⇨ In-kernel FPU APIs:
  - **`irq_fpu_usable()`**: restrictive
  - **`kernel_fpu_begin()/kernel_fpu_end()`**: non-preemptible
  - Assembly-based limited usage
- Entry ⇨ No API
  - Compiler-based scaled application
  - Use everywhere, but increase image size

Adoption Trade-offs

Scope	Limited	Wider
Tool	Assembler	Compiler
API: Usability	Restrictive	Unrestrictive
API: Preemptibility	No	Yes
Context switch	`XSAVE`	Entry
Image size increase	Manageable	Large

Rethink APIs

- Existing non-preemptible APIs

  ⇨ Limited adoption

- Opportunity: relax via preemption support
  - Kernel EGPR state saved on preemption
    - Another **XSAVE** buffer?
    - But **XSAVE** buffer - 576B+, EGPRs - 128B
  - Maybe, just save EGPR state only?

XSAVE Area	**Size (Bytes)**
Legacy states	512
XSAVE header	64
APX state: EGPRs	16 x 8

Closing

- Unknowns: measured data to demonstrate APX benefits
- Considerable options:
  - Limited scope with existing APIs
  - Wider adoption with increased image size
  - Or, rethink APIs to find some middle ground
- Next steps?

Discussions

Alternative API: Expectations

	Limited	Wide (alt. API)	Wider
Tool	Assembler	Compiler	Compiler
API: Usability	Restrictive	Restrictive	Unrestricted
API: Preemptibility	No	Yes	Yes
Context switch	`XSAVE`	`XSAVE` + EGPR buffer	Entry
Image size increase	Manageable	Depends on	Large

- Either extend existing APIs or new EGPR-specific ones
- Maybe, fall between the two models

